

✦ 小白也能看懂的开源项目解读 ✦

img2threejs 入门指南

一张图片，如何被 AI 变成一个纯代码的 3D 模型？

这个项目做的事，一句话概括：你给它一张物体的照片，它指挥 AI 编程助手，用「写代码」的方式把照片里的东西重新「捏」成一个 3D 模型——模型由代码生成、在浏览器里实时渲染、可以直接旋转查看，甚至可以绑定骨骼做动画。

📄 Apache-2.0 开源免费

🐍 纯 Python 标准库，零安装依赖

🌐 Three.js 网页 3D

🖼️ 图片 → 代码 → 3D

💻 配合 Claude Code / Codex / OpenCode 使用

项目地址: github.com/img2threejs/img2threejs | 在线演示画廊: img2threejs.io | 本地克隆位置:
F:\reminder\img2threejs

本指南基于项目 v2.0.0 的 README、ARCHITECTURE 等官方文档整理，面向零基础读者。

📖 目录 · 小白的 12 个疑问

- 01 这个项目到底是干什么的？
- 02 "用代码捏模型"和我听过的其他 3D 建模方式有什么不一样？
- 03 我是小白，怎么把它跑起来？需要会编程吗？
- 04 一张照片是怎么一步步变成 3D 模型的？
- 05 Three.js 是什么？为什么模型要"在浏览器里跑"？
- 06 项目里一堆术语——Skill、Spec、Gate、Pass、Rig、GLB.....都是什么意思？
- 07 跑完之后，我能拿到什么东西？能拿来干什么实际的事？
- 08 它是免费的吗？会不会很烧钱（烧 Token）？
- 09 它做不到的事情有哪些？结果能 100% 像照片吗？
- 10 我想要一个"会动的角色"，可以吗？
- 11 想看懂它的代码，应该从哪里入手？
- 12 遇到问题、想深入学习，去哪里找资料？

Q1 这个项目到底是干什么的？

想象你有一张 **CS2 刀具**、一辆 **BMX 自行车** 或一只 **皮卡丘** 的图片。img2threejs 会让你常用的 AI 编程助手（比如 Claude Code）分析这张图，然后**写出一堆 TypeScript 代码**——这些代码运行时会“凭空搭建”出一个 3D 模型：用球体、圆柱、自定义曲面等基本形状拼出外形，用程序化材质还原颜色和质感。

关键点：最终产物**不是**一个几 MB 的模型文件（.obj / .glb / .fbx），而是一份**可阅读、可修改、可版本管理的代码文件**（例如 `createTalonDopplerRubyModel.ts`）。把它放进网页，任何人打开浏览器就能看到一个可以拖拽旋转的 3D 模型。



图 1 · img2threejs 的一句话 workflow：图片 → 代码 → 浏览器里的 3D 模型

✅ **一个直观的入口：**先别读文档，直接打开官方在线演示画廊 img2threejs.io ——画廊里的每个模型（双剑女战士、低多边形人形、CS2 刺刀、自行车、皮卡丘……）**全部是代码生成的**，你可以旋转查看、对照参考图、甚至点开看生成它的源码。

Q2 "用代码捏模型"和其他 3D 建模方式有什么不一样？

获得一个 3D 模型常见有三种路子，img2threejs 选了第三种：

方式	① 拍照扫描（摄影测量）	② AI 网格生成（如 Tripo、Rodin）	③ 代码重建（本项目）
怎么得到	围着物体拍几百张照片，软件拼出模型	把图喂给 AI 服务，几秒吐出一个网格文件	AI 看图后 写代码 ，代码运行时生成模型
产出物	大体积网格文件	GLB/FBX 等 mesh 文件	纯文本代码 + 一份 JSON 规格书
能不能改	很难改，像"拍死的"雕塑	可以微调，但整体是"一坨"	改个数字就改形状，像改菜谱
能不能动画	要额外绑骨骼，很费劲	看情况，常需要重拓扑	天生带关节轴心、挂点、碰撞体， 为动画而生
体积	大	中	很小（就是文本）

代码重建的独特好处：模型是"活的菜谱"而不是"死的雕塑"。想给刀身换个颜色？改一个数字。想让模型轻一点？删一段代码。两个版本之间有什么不同？用 Git 对比文本一目了然。这正是为什么它的产物对**游戏开发和网页展示**特别友好。

⚠ 代价也要知道：代码重建是"程序化近似"，不是像素级复刻。它擅长硬表面物体（刀具、枪械、载具、小物件）；对照片级真实人脸这类目标，它只能做风格化重建（详见 Q9）。

Q3 我是小白，怎么把它跑起来？需要会编程吗？

先说结论：**不需要你会写代码，但你需要一个"AI 编程助手"替你跑**。这个项目本身其实是一个"技能包" (Skill) + 一堆 Python 脚本，它的正确打开方式是装进 Claude Code、Codex 或 OpenCode 里，然后用一句话指挥它。

三步走：

```
// 第 1 步：把项目装进 AI 助手的技能目录（一行命令） git clone  
https://github.com/img2threejs/img2threejs.git ~/.claude/skills/img2threejs // 第 2 步（可选）：  
安装领域插件，比如 CS2 枪械皮肤重建 npx github:img2threejs/img2 install img2 add  
img2threejs/plugin-cs2 // 第 3 步：在 Claude Code 里，附上一张物体图片，说一句话： /img2threejs 把  
这张图里的物体重建为 Three.js 模型，保持比例、角度和颜色。
```

说完这句话，剩下的它全包了：检查图片 → 分析细节 → 写规格书 → 一轮轮生成代码 → 渲染出来和原图对比 → 不像就自己改，直到达标。

✅ **环境要求低得感人**：所有脚本只用 Python 3.10+ 的**标准库**——不用 pip 装任何东西，不用装 numpy、PIL，一个依赖都没有。有 Python 就能跑（你克隆到的 `F:\reminder\img2threejs` 就是一份完整副本）。

⚠️ **小白最容易卡住的一步**：第 1 步里的 `~/.claude/skills/` 是 Claude Code 的技能目录。如果命令报错找不到路径，直接把克隆下来的文件夹放进 `C:\Users\你的用户名\.claude\skills\` 下面即可。

Q4 一张照片是怎么一步步变成 3D 模型的？

整个流程像一条**带质检员的流水线**：AI 负责动手和“看”，Python 脚本负责把关。每一步没通过质检，就不能进入下一步；渲染出来不像，就打回重做。

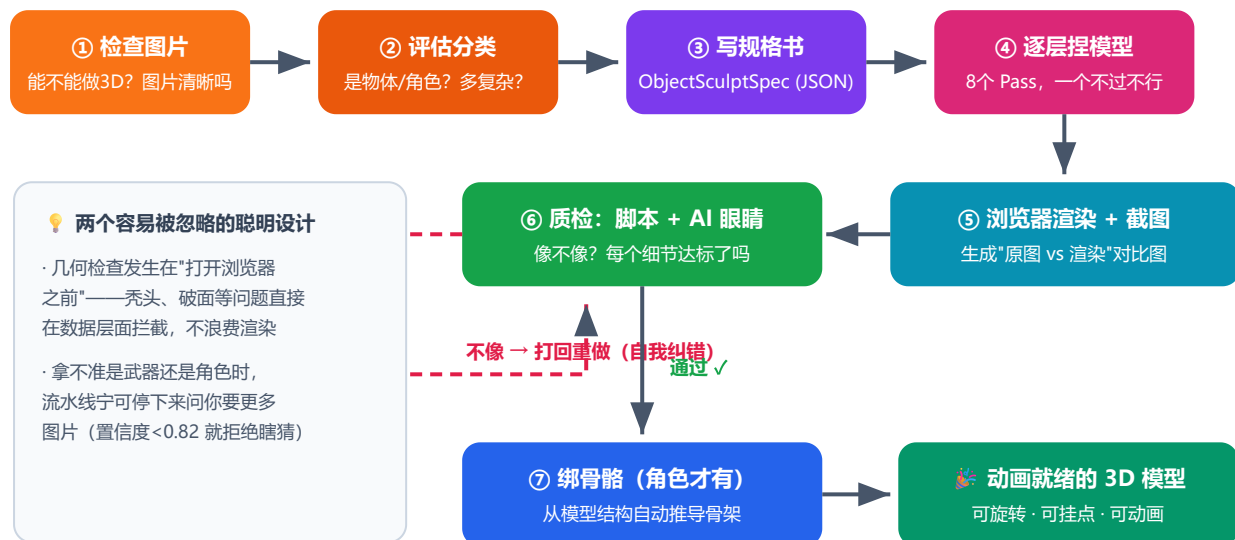


图 2 · 从图片到模型的完整流水线（依据 docs/ARCHITECTURE.md 绘制）

8 个“捏模型”的 Pass 按固定顺序进行，上一层验收通过才解锁下一层——很像画画的“起稿→结构→形体→上色→质感→打光”：



每个 Pass 必须有：真实渲染 + 对比图 + AI 视觉评分达标，才能进入下一个

图 3 · 八层构建 Pass：从粗糙块面到最终优化

特别值得一提的是“细节清单”（detailInventory）：生成代码之前，AI 必须先把照片里所有**身份级小细节**列出来——倒角、螺丝铆钉、刻线、磨损、高光哑光的分区……每一个细节都必须对应到真实部件上，**列不全就不许开始写代码**。这是它成图“像”的关键。

Q5 Three.js 是什么？为什么模型要"在浏览器里跑"？

Three.js 是世界上最流行的 JavaScript 3D 库：网页通过它调用显卡（WebGL），把 3D 场景画进浏览器。你在网页里看到的很多 3D 展示、数据可视化、小游戏，背后都是它。

img2threejs 生成的代码，本质是一个函数：

`createObjectNameModel(spec, options)` ——调用它就返回一个 `THREE.Group`（一组 3D 对象的容器）。也就是说，**模型 = 一段可执行的搭建说明**。

为什么坚持浏览器渲染？ 因为质检环节需要"眼睛"：模型渲染出来、截屏、和原图并排对比，AI 才能判断像不像。浏览器就是现成的、免费的渲染器，还天然跨平台。

副产品是：你得到的东西天然适合**网页展示和分享**——发给朋友一个链接就能看，不用装任何软件。

✅ **小白记忆法：**把 Three.js 想成"网页版的乐高说明书引擎"，img2threejs 生成的代码就是那份说明书，浏览器负责照着说明书把乐高拼出来给你看。

Q6 项目里一堆术语——Skill、Spec、Gate、Pass、Rig、GLB.....都是什么意思？

别怕，核心术语其实就这几个，每个一句话：

Skill 技能包

这个项目整个就是一个 Claude Code"技能"：装进技能目录后，AI 助手就学会了一门手艺（图片转 3D），用 `/img2threejs` 一句话召唤。

ObjectSculptSpec 规格书

一份 JSON 文件，完整描述要捏的东西：零件树、每个零件的形状/位置/材质、活动关节、挂点.....相当于**施工图纸**。代码是照着它生成的，改图纸就能改模型。

Gate 质检关卡

每一步的"考官"。比如"秃头检测"（角色头发不能漏头皮）、"左右手性关卡"（左手不能是复制来的右手）。**考试不过，禁止进入下一步**，从制度上防止 AI 糊弄。

Pass 构建层

见 Q4 的八层流水线。AI 每次只写"当前解锁的那一层"的代码，不重复读整个模型——这也是省钱的关键。

Forge / Grimoire

仓库里两大目录的代号：**forge/**（锻造厂）= 约 90 个确定性 Python 工具脚本；**grimoire/**（魔法书）= 各道关卡使用的评分标准和方法论文档。

Rig 骨骼绑定

给角色模型装上"骨头"（骨架），把皮肤"挂"在骨头上，骨头动皮肤跟着动——角色才能做动作。本项目会从模型零件树自动推导骨架。

GLB

一种通用 3D 文件格式。本项目**不产出** GLB；但可选地，可以把一个 GLB 当"卷尺"用来量尺寸（量完即扔，不进入成品）。另有插件可以把图发给外部 AI（如 TRELIS）生成 GLB。

PBR

"基于物理的渲染"——让模型表面在光线下表现真实（金属反光、粗糙哑光等）的材质体系。项目会从照片取色分析，反推每个区域的粗糙度、金属感等参数。

Token

AI 模型计费/计量的"字数"。AI 读图、思考、写代码都在消耗 token——见 Q8，这个项目的一大卖点就是省 token。

Q7 跑完之后，我能拿到什么东西？能干什么实际的事？



图 4 · 一次运行拿到什么 · 它能帮你做什么

产出是"四件套"：规格书 JSON、TypeScript 模型工厂、角色的骨骼数据、以及记录每一轮保真度的对比图。其中模型工厂暴露了 `root.userData.sculptRuntime` ——里面有权轴 (pivot)、挂点 (socket)、碰撞体 (collider)，意思是这模型**生下来就是为了被程序操控的**：门能开、刀能挥、轮子能转。

实际用途时间线：现在就能用于网页 3D 展示和游戏原型；按官方路线图，v2.3 会加入 Unity/Unreal 导出器和 Blender 桥接，v2.5 做网页 UI 和批处理，v3.0 目标是"从参考图生成可玩世界"。

Q8 它是免费的吗？会不会很烧钱（烧 Token）？

项目本身完全免费开源（Apache 2.0 协议），连赞助都是自愿的（Ko-fi）。你克隆到本地不花一分钱。

真正花钱的地方：驱动它的 AI 编程助手按用量计费——AI 每次看图、思考、写代码都消耗 token。一个物体要经过 8 层 Pass、多轮“渲染-对比-修正”循环，这个流程**天生是吃 token 的**（README 原话：“every gate rerun ... spends tokens”）。

但作者把“省钱”做成了核心设计，这也是这个项目最值得学的思路：

- 💰 **五个省 Token 的机制**：
- ① **脚本干活，AI 判断**——校验、打包、记录全是 Python 的事，AI 只做一件事：看一张对比图，说“过/不过”；
 - ② **零依赖**——纯标准库意味着不会在环境问题上浪费对话轮次；
 - ③ **逐层生成**——每轮只写当前层的代码，不反复读整个模型；
 - ④ **先质检再生成**——规格书太浅直接拦下，不浪费一次渲染；
 - ⑤ **文字产物**——代码 + JSON 都是小文本，不是几 MB 的网格文件。

另外，仓库列了三家赞助商（Atlas Cloud、Tripo、Hyper3D）都是“图生 3D”或推理平台——它们的存在本身就在提示你：跑这条流水线的算力开销是真实的，作者用赞助覆盖它。

Q9 它做不到的事情有哪些？结果能 100% 像照片吗？

这个项目有个难得的优点：**诚实**。README 里专门有一节"Honesty about limits"，总结下来：

✖ 做不到 / 弱项	✔ 它的应对方式
单张照片看不到背面	用可见面的镜像来推断被挡住的部分，并明确标注"这是推断，低置信度"，绝不假装确定
照片级真人相似度	明说：角色是"风格化重建"，不是照片级复刻；提供了可选的"最大化相似度"路径，但会告诉你每个区域有几成把握，不够就管你要更多照片
图片本身质量差	流水线第一步就是"适检关卡"——图不行会直接拒绝，而不是硬着头皮做出个四不像
贴图靠"偷"照片	代码重建的契约是 不下载、不贴图 ——材质质感全部由代码参数生成；只有可选的"投影路线"会把照片投影到模型上，且会做去光照处理并明确标注来源
"达不到要求的保真度"	项目原话：这是一个 合法且预期内的输出结果 。宁可明说做不到，也不硬凑

⚠ **给小白的心理预期：** 把它的产出理解为"一个还原度很高、结构正确、可以旋转可以动画的风格化模型"，而不是"照片的 3D 复印件"。想要照片级复刻，路线图里的多视角重建（v3.0）才是目标方向。

Q10 我想要一个"会动的角色", 可以吗?

可以, 这正是 v1.5"角色更新"的主题, 但要做对几件事:

- ① **角色走专属流水线**: 图会被识别为 `character` 类, 进入"解剖学感知"通道——头身比例、五官定位、姿势分析都有专门关卡 (比如前面说的"秃头检测", 是在几何数据上直接查的, 根本不等渲染)。
- ② **自动绑骨骼**: `stage5_rig` 从模型的零件树推导骨架 (骨头不会脱离几何乱跑), 用"测地距离"计算每块皮肤归哪根骨头管 (手抬起来, 前臂的皮肤跟着动, 肩膀不动)。绑好后模型带 `root.userData.rig`, 供 Three.js 的动画系统驱动。
- ③ **想要现成的动画角色, 装插件**: `img2 add img2threejs/plugin-character` 提供 `animated-character` 档位, 加入绑骨/动画的强制关卡。头发也是独立五阶段子系统 (发际线、高光带、发根到发梢的渐变都有专门检查)。
- ④ **更完整的动画能力在路线图上**: 自动绑骨、Mixamo 兼容、面部表情骨骼在 v2.4"动画更新"。目前版本动画能力是"骨架就绪", 离"一键跳舞"还有距离——这也是看路线图判断项目成熟度的好例子。

✅ **想直接看效果**: 演示画廊里的"低多边形绑骨人形"和"双剑女战士"就是角色路线的产出, 可以在线旋转把玩。

Q11 想看懂它的代码，应该从哪里入手？

📁 F:\reminder\img2threejs —— 仓库地图（小白游览顺序）

- 1 **README.md** 项目门面：能干嘛、怎么装、演示画廊链接（5 分钟）
- 2 **docs/ARCHITECTURE.md** 精华！流水线全景图 + 每道关卡的原理（30 分钟）
- 3 **forge/** 约 90 个 Python 脚本，按 stage1~5 分目录——流水线的“机械部分”，读脚本名就能懂流程
- 4 **grimoire/** 评分标准与方法论（“AI 的审美是怎么定的”）
- 5 **SKILL.md** 给 AI 看的“操作手册”——学提示词工程的绝佳教材
- 6 **ROADMAP.md + CHANGELOG.md** 版本演进史：看一个项目怎么从 v1.0 走到 v2.0
- 7 **forge/tests/** 测试用例：想知道某脚本干嘛的，读它的测试最快

图 5 · 仓库游览路线：按 1→7 的顺序读，从游客到进阶

读代码前先记住仓库的设计哲学（CLAUDE.md 里写明的）：`forge/` 是确定性工具（同样的输入永远同样的输出），`grimoire/` 是给 AI 的参考资料，“脚本负责强制执行，模型只负责审美判断”。理解了这句，整个仓库的结构就通了。

另一个值得学的点：这个项目怎么写文档。它给人类看 README / ARCHITECTURE，给 AI 看 SKILL.md / grimoire/，还专门写了 CLAUDE.md 约束 AI 助手的行为规则——“人类文档”和“AI 文档”分层管理，这是 AI 时代开源项目的新形态。

想上手改代码：读 CONTRIBUTING.md，跑 `python3 -m unittest discover -s forge/tests` 验证环境（Windows 上把 `python3` 换成 `python`）。作者特别欢迎的贡献方向：程序化材质配方、新关卡、新演示。

Q12 遇到问题、想深入学习，去哪里找资料？

我想.....	去这里
亲眼看看效果	在线演示画廊 img2threejs.io (每个 demo 可看源码)
搞懂工作原理	docs/ARCHITECTURE.md (流水线全景)
查某个脚本怎么用	grimoire/scripts.md (全部脚本的可执行参考)
了解每道质检关卡的标准	grimoire/review/gates_reference.md
重建一个角色	grimoire/character/reconstruction.md
估算一次要花多少 token	docs/TOKEN_COST.md
了解项目未来计划	ROADMAP.md (直到 v3.0 的规划)
报 bug / 提功能	GitHub Issues; 贡献指南在 CONTRIBUTING.md
写自己的领域插件	img2 仓库的 docs/WRITING_A_PLUGIN.md

✅ **学习路径建议 (按顺序)**：① 打开演示画廊玩 10 分钟，建立直观印象 → ② 读本指南 Q1–Q6，理解“图→代码→模型”的核心概念 → ③ 装进 Claude Code，拿一个简单物体（比如一把剪刀、一个水杯）试跑一次 → ④ 读 ARCHITECTURE.md 对照刚才的体验复盘 → ⑤ 进阶：读 SKILL.md 学它怎么“指挥 AI”，或挑一个演示的源码看 AI 写的 3D 代码长什么样。

★ 本指南由 Claude Code 基于仓库官方文档 (README.md · docs/ARCHITECTURE.md · CLAUDE.md · ROADMAP.md, 版本 v2.0.0) 整理生成 · 仓库已克隆至 F:\reminder\img2threejs · 所有观点以官方文档为准 · 图表均为原创示意图